



The following versions used in the simulations.

- heads/master-dirty-ce88a576295ce68f40736599a9e3e04f24f38801(0.6.3)
- heads/alex-dev-defe967c1db10e3b873fe074f274b6bdc9c42bb8(0.6.3)

```
Configuration{
  random_seed: ![ 42, 935, 128643 ],
  warmup: 20000,
  measured: 10000000000000000,
  general_frequency_divisor: 2,
  topology: topologia![
    MultiStage{
      stages: [
        ExplicitStageFile{ filename: "../MRLS_1280_19_760_32_a_net" },
        servers_per_leaf: 13,
        legend_name: "imported MRLS cost 1.46",
        Megafly{ global_ports_per_spine: 16, servers_per_leaf: 16, global_arrangement: Palmtree, group_size: 16, number_of_groups: 65, legend_name:
"Dragonfly+" },
        traffic: ![
          TrafficMap{
            tasks: 16640,
            map: ![
              Composition{
                patterns: [
                  Identity{ legend_name: "Natural" },
                  CartesianEmbedding{
                    source_sides: [ 16384 ],
                    destination_sides: [ 16640 ]}],
                    middle_sizes: [ 16384, 16640 ],
                    legend_name: "Natural"}],
                    application: AllReduce{ tasks: 16384, data_size: 16384 },
                    legend_name: "AllReduce"},
          TrafficMap{
            tasks: 16640,
            map: ![
              Composition{
                patterns: [
                  Identity{ legend_name: "Natural" },
                  CartesianEmbedding{
                    source_sides: [ 16512 ],
                    destination_sides: [ 16640 ]}],
                    middle_sizes: [ 16512, 16640 ],
                    legend_name: "Natural"}],
                    application: ![
                      All2All{ tasks: 16512, data_size: 132096 }],
                      legend_name: "All2All"}],
          maximum_packet_size: 16,
        router: InputOutput{
          virtual_channels: 4,
          virtual_channel_policies: topologia![
            [
              MapHop{
                hop_to_policy: [
                  ArgumentVC{
                    allowed: [ 0 ]},
                  ArgumentVC{
                    allowed: [ 0 ]},
                  ArgumentVC{
                    allowed: [ 1 ]},
                  ArgumentVC{
                    allowed: [ 1 ]},
                  ArgumentVC{
                    allowed: [ 2 ]},
                  ArgumentVC{
                    allowed: [ 2 ]},
                  ArgumentVC{
                    allowed: [ 3 ]},
                  ArgumentVC{
                    allowed: [ 3 ]}],
                  VecLabel{
                    label_vector: [ 0, 64, 80 ]},
                  OccupancyFunction{ label_coefficient: 1, occupancy_coefficient: 1, product_coefficient: 0, constant_coefficient: 0, use_internal_space: true,
use_neighbour_space: true, aggregate: true },
                  OccupancyFunction{ label_coefficient: 1, occupancy_coefficient: 1, product_coefficient: 0, constant_coefficient: 0, use_internal_space: true,
use_neighbour_space: true, aggregate: false },
                  LowestLabel,
                  EnforceFlowControl,
                  Random],
            [
              MapHop{
                hop_to_policy: [
                  MapLabel{
                    label_to_policy: [
                      Chain{
                        policies: [
                          OccupancyFunction{ label_coefficient: 0, occupancy_coefficient: 1, product_coefficient: 0, constant_coefficient: 0,
use_internal_space: true, use_neighbour_space: true, aggregate: false }],
                        Chain{
                          policies: [
                            AverageOccupancyFunction{
                              label_coefficient: 0,
                              occupancy_coefficient: 2,
                              product_coefficient: 0,
                              constant_coefficient: 16,
                              use_internal_space: true,
                              use_neighbour_space: true,
                              exclude_minimal_ports: false,
                              virtual_channels: [ 0 ]}]},
                            MapLabel{
                              label_to_policy: [
                                Chain{
                                  policies: [
                                    OccupancyFunction{ label_coefficient: 0, occupancy_coefficient: 1, product_coefficient: 0, constant_coefficient: 0,
```

```

use_internal_space: true, use_neighbour_space: true, aggregate: false }},
Chain{
  policies: [
    AverageOccupancyFunction{
      label_coefficient: 0,
      occupancy_coefficient: 3,
      product_coefficient: 0,
      constant_coefficient: 16,
      use_internal_space: true,
      use_neighbour_space: true,
      exclude_minimal_ports: false,
      virtual_channels: [ 0 ],
      exclude_link_classes: [ 0 ]}}}],
above_policy: MapLabel{
  label_to_policy: [
    Chain{
      policies: [
        OccupancyFunction{ label_coefficient: 0, occupancy_coefficient: 1, product_coefficient: 0, constant_coefficient: 0, use_internal_space:
true, use_neighbour_space: true, aggregate: false }},
Chain{
      policies: [
        OccupancyFunction{ label_coefficient: 0, occupancy_coefficient: 1, product_coefficient: 0, constant_coefficient: 48, use_internal_space:
true, use_neighbour_space: true, aggregate: false }}}}],
LowestLabel,
EnforceFlowControl,
Random]],
allocator: Random,
buffer_size: 128,
bubble: false,
flit_size: 16,
intransit_priority: false,
allow_request_busy_port: true,
output_buffer_size: 64,
crossbar_frequency_divisor: 1,
crossbar_delay: 2},
routing: topologia![
  Polarized{ include_labels: true, strong: false, legend_name: "Polarized" },
  MegaflyAD{
    first_allowed_virtual_channels: [ 0, 1 ],
    second_allowed_virtual_channels: [ 2, 3 ],
    minimal_to_deroute: [ 1, 1, 0 ],
    source_group_pattern: [
      [ None, None ],
      [ None, None ]],
    intermediate_group_pattern: [
      [ None, None ],
      [ None, None ]],
    destination_group_pattern: [
      [ None, None ],
      [ None, None ]],
    global_pattern_per_hop: [
      [ None, None ],
      [ None, None ]],
    legend_name: "FPAR 2 vc"}],
link_classes: [
  LinkClass{ delay: 2 },
  LinkClass{ delay: 2 },
  LinkClass{ delay: 2 }],
launch_configurations: [
  Slurm{ job_pack_size: 1, time: "3-00:00:00" }}]

```